Agora Livestream Integration

Overview

Agora handles media transport, while Yenkasa controls:

- who can start a stream
- who can join as broadcaster or audience
- how UIDs are derived
- how tokens are minted and expired

Current implementation

Token generation

- file: `utils/agoraTokenGenerator.js`
- validates:
  - `AGORA_APP_ID`
  - `AGORA_APP_CERTIFICATE`
- derives a deterministic numeric Agora UID from the Mongo user id
- returns role-specific tokens for broadcaster or subscriber

Stream join flow

- REST route issues the token
- Socket.IO confirms host readiness and room membership
- the live model stores channel and guest/broadcaster state

Important properties

- deterministic UID mapping helps keep backend and clients aligned
- token TTL defaults to four hours
- invalid or missing Agora config fails fast with explicit error codes

Known issues

- Agora configuration failures are handled cleanly, but operational discovery st
ill depends on logs
- livestream media transport and livestream room semantics are tightly coupled i
n the same backend

Scaling concerns

- token generation itself is cheap
- the risk is not Agora token issuance; it is the surrounding room and lifecycle
 orchestration

Recommended improvements

1. expose Agora health and credential readiness through an admin health endpoint
2. version the livestream join contract explicitly
3. decouple token issuance from broader livestream orchestration over time